

Linguaggi Formali e Compilatori

(Formal Languages and Compilers)

prof. S. Crespi Reghizzi, prof. Angelo Morzenti
(prof. Luca Breveglieri)

Prova scritta - 24 settembre 2008 - Parte I: Teoria

CON SOLUZIONI - A SCOPO DIDATTICO LE SOLUZIONI SONO MOLTO ESTESE E COMMENTATE VARIAMENTE - NON SI RICHIEDE CHE IL CANDIDATO SVOLGA IL COMPITO IN MODO ALTRETTANTO AMPIO, BENSÌ CHE RISPONDA IN MODO APPROPRIATO E A SUO GIUDIZIO RAGIONEVOLE

NOME:

COGNOME:

MATRICOLA:

FIRMA:

ISTRUZIONI - LEGGERE CON ATTENZIONE:

- L'esame si compone di due parti:
 - I (80%) Teoria:
 1. espressioni regolari e automi finiti
 2. grammatiche libere e automi a pila
 3. analisi sintattica e parsificatori
 4. traduzione sintattica e analisi semantica
 - II (20%) Esercitazioni Flex e Bison
- Per superare l'esame l'allievo deve sostenere con successo entrambe le parti (I e II), in un solo appello oppure in appelli diversi, ma entro un anno.
- Per superare la parte I (teoria) occorre dimostrare di possedere conoscenza sufficiente di tutte le quattro sezioni (1-4), rispondendo alle domande obbligatorie.
- È permesso consultare libri e appunti personali.
- Per scrivere si utilizzi lo spazio libero e se occorre anche il tergo del foglio; è vietato allegare nuovi fogli o sostituirne di esistenti.
- Tempo: Parte I (teoria): 2h.30m - Parte II (esercitazioni): 45m

1 Espressioni regolari e automi finiti 20%

1. È dato l'alfabeto $\Sigma = \{a, b, c\}$. Si considerino le due espressioni regolari R_1 e R_2 seguenti, di alfabeto Σ :

$$R_1 = a (a b \mid b c)^*$$

$$R_2 = (a b^* \mid a c^*)^*$$

e i rispettivi linguaggi regolari L_1 e L_2 .

Si risponda alle domande seguenti:

- (a) Si dica quali sono i digrammi contenuti nelle stringhe dei linguaggi L_1 e L_2 , rispettivamente.
- (b) Si dica se le espressioni regolari R_1 e R_2 sono ambigue, con motivazione (ossia se l'espressione non è ambigua si spieghi perché, se lo è si dia una stringa ambigua).
- (c) Si dica se i linguaggi L_1 e L_2 sono locali, motivando la risposta.
- (d) (facoltativa) Si trovi l'espressione regolare R_3 (contenente i soli operatori di concatenamento unione stella croce) che genera il linguaggio regolare intersezione $L_3 = L_1 \cap L_2$, lavorando a scelta in modo intuitivo o algoritmico.

Soluzione

- (a) Un modo per trovare i digrammi è di ricorrere al calcolo dei séguiti. Prima si numerano i generatori nelle espressioni regolari R_1 e R_2 , come segue:

$$R_1 = a_1 (a_2 b_3 \mid b_4 c_5)^*$$

$$R_2 = (a_1 b_2^* \mid a_3 c_4^*)^*$$

I séguiti sono i seguenti:

espressione R_1		espressione R_2	
generatore	seguito	generatore	seguito
a_1	a_2	a_1	a_1
a_2	b_3	a_1	a_3
b_3	a_2	a_1	b_2
b_3	b_4	b_2	a_1
b_4	c_5	b_2	a_3
c_5	a_2	b_2	b_2
c_5	b_4	a_3	a_1
		a_3	a_3
		a_3	c_4
		c_4	a_1
		c_4	a_3
		c_4	c_4

Trascurando la numerazione si ottengono i digrammi seguenti:

digrammi di $R_1 = a a \quad a b \quad b a \quad b b \quad b c \quad c a \quad c b$

digrammi di $R_2 = a a \quad a b \quad a c \quad b a \quad b b \quad c a \quad c b \quad c c$

(b) L'espressione regolare R_1 è non ambigua. Osservando la numerazione, si ha che:

- il generatore a_1 compare all'inizio dell'espressione, deve necessariamente generare il carattere iniziale della stringa, e dunque non dà luogo ad ambiguità
- trascurando la a iniziale (già discussa e innocua ai fini dell'ambiguità), le lettere a e c compaiono in posizioni uniche nell'espressione, e dunque non possono essere generate ambigualmente in nessuna stringa
- la lettera b compare in due posizioni differenti nell'espressione, e dunque quando la stringa ne contiene una, ci si potrebbe chiedere quale delle due comparse di b la ha generata; tuttavia i generatori b_3 e b_4 sono sempre distinguibili in quanto b_3 è preceduto da a e b_4 è seguito da c ; dunque la lettera b non può essere generata ambigualmente in nessuna stringa perché si distingue sempre guardando la lettera che precede o segue

In conclusione l'espressione R_1 non può generare la stessa stringa in due o più modi diversi, e pertanto non è ambigua.

L'espressione regolare R_2 è ambigua. Osservando la numerazione, si vede infatti che la stringa a è generata in due modi come a_1 e a_3 . Più in generale, le stringhe a^+ sono tutte ambigue, in quanto generate come $(a_1 \mid a_3)^+$, sottoespressione derivata da R_2 e molto ambigua. Si ha infatti che il grado di ambiguità di a^n ($n \geq 1$) vale 2^n , addirittura esponenziale (si lascia la verifica al lettore).

(c) Il linguaggio L_1 non è locale: stando ai digrammi $a a$ e $a b$ di R_1 , si dovrebbe avere $a a a b \in L_1$, ma R_1 non genera tale stringa.

Il linguaggio L_2 è locale. Infatti l'espressione R_2 si riscrive così, con raccoglimento:

$$R_2 = (a b^* \mid a c^*)^* = (a (b^* \mid c^*))^*$$

La forma riscritta è di tipo lineare (ogni generatore compare una sola volta), e pertanto il linguaggio L_2 è locale.

- (d) Ricorrendo all'intuizione, si trova rapidamente l'intersezione R_3 delle due espressioni R_1 e R_2 . Si cominci con il porre la formulazione seguente di R_3 :

$$R_3 = (a (a b \mid b c)^*) \cap (a b^* \mid a c^*)^*$$

da dove però bisogna eliminare l'operatore di intersezione. Se nel secondo membro dell'intersezione l'elemento c^* viene derivato e genera una o più c , si forma il digramma ac , che il primo membro non è in grado di produrre. Pertanto nel secondo membro l'elemento c^* deve sempre derivarsi in ε . Si ha dunque la semplificazione seguente:

$$R_3 = (a (a b \mid b c)^*) \cap (a b^* \mid a)^*$$

Ora però il secondo membro dell'intersezione non contiene (più) il generatore c , e dunque neppure il primo lo può derivare. Giacché c compare nel secondo componente bc dell'unione, si può semplicemente rimuovere tale componente, e di conseguenza l'unione. Si ha dunque la semplificazione seguente:

$$R_3 = (a (a b)^*) \cap (a b^* \mid a)^*$$

che ovviamente si riduce ulteriormente così (giacché a è contenuto in $a b^*$):

$$R_3 = (a (a b)^*) \cap (a b^*)^*$$

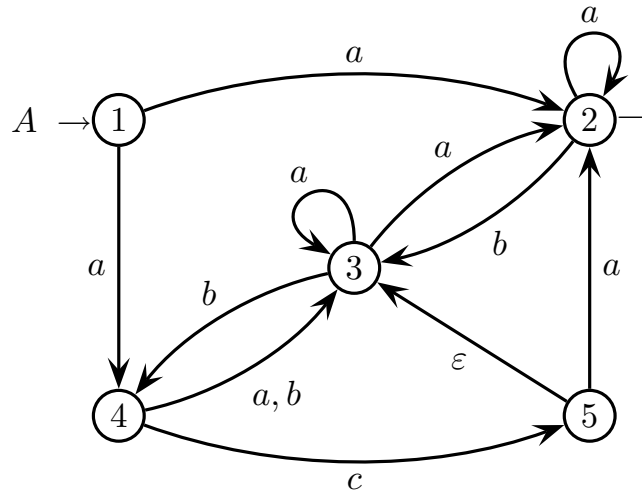
Così l'espressione R_3 si è alquanto ridotta e si possono confrontare direttamente i due membri dell'intersezione. Il primo genera stringhe di tipo $aab \dots ab$ e il secondo di tipo $ab \dots b \dots ab \dots b$ (il secondo genera il linguaggio universale di alfabeto $\{a, b\}$ dove ogni stringa tranne ε inizia con a). Evidentemente il primo è un sottinsieme del secondo, dunque si ha la semplificazione finale:

$$R_3 = (a (a b)^*)$$

che rappresenta l'espressione regolare R_3 cercata. Si vede pure facilmente che questa formulazione di R_3 è non ambigua.

In alternativa si può ricorrere ai metodi algoritmici: trasformare R_1 e R_2 in automi (det. o anche non det.) tramite l'algoritmo di Berri-Sethi, trovare l'automa intersezione tramite prodotto di automi, e ritrasformare l'automa intersezione in espressione regolare tramite il metodo di eliminazione dei nodi (Brozowski), o altri affini; il procedimento arriva senz'altro in porto e non richiede grandi doti d'intelletto, ma è un po' lungo e tedioso da svolgere.

2. È dato l'automa non deterministico A mostrato in figura, di alfabeto $\{a, b, c\}$.



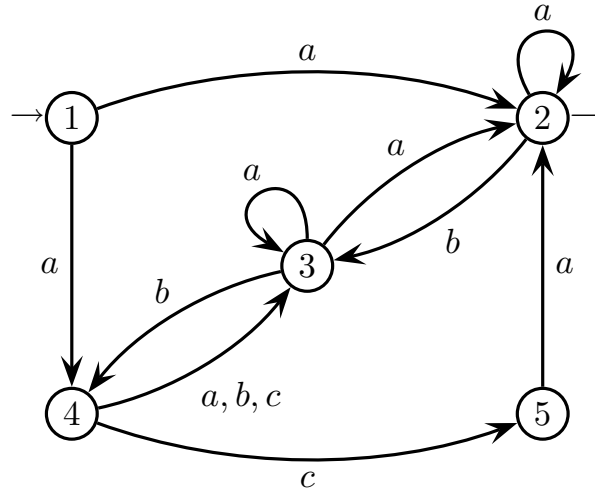
Si risponda alle domande seguenti:

- Si costruisca un automa deterministico A' equivalente ad A e se ne tracci il grafo stato-transizione.
- Se necessario, si trovi l'automa minimo A'' equivalente ad A' e se ne tracci il grafo stato-transizione.
- (facoltativo) Si trovi un'espressione regolare R , preferibilmente non ambigua, che genera il linguaggio $L(A)$.

Soluzione

- Si osservi che l'automa A è indeterministico in quanto ha una transizione spontanea e archi con etichette duplicate.

Si procede dapprima all'eliminazione della transizione spontanea $5 \xrightarrow{\varepsilon} 3$: gli archi entranti nello stato 5 vanno duplicati nello stato 3. Ecco il risultato:



Ora si può applicare la costruzione classica dei sottinsiemi, tramite la tabella dei successori:

gruppo	a	b	c	finale?
1	24	—	—	no
24	23	3	3	sì
23	23	34	—	sì
3	23	4	—	no
34	23	4	35	no
4	3	3	35	no
35	23	4	—	no

Rinominando i gruppi di stati, si ottiene la tabella degli stati seguente:

stato	a	b	c	finale?
α	β	—	—	no
β	γ	δ	δ	sì
γ	γ	ε	—	sì
δ	γ	ζ	—	no
ε	γ	ζ	η	no
ζ	δ	δ	η	no
η	γ	ζ	—	no

Questa è la tabella degli stati dell'automa deterministico A' . Si lascia al lettore il compito di tracciarne il grafo stato-transizione.

- (b) Riprendendo la tabella degli stati dell'automa deterministico A' , si vede subito che le righe δ e η sono identiche, pertanto gli stati δ e η sono indistinguibili. La tabella si semplifica nel modo seguente:

stato	a	b	c	finale?
α	β	—	—	no
β	γ	δ	δ	sì
γ	γ	ε	—	sì
δ	γ	ζ	—	no
ε	γ	ζ	δ	no
ζ	δ	δ	δ	no

Questa tabella degli stati è in forma minima, perché:

- gli stati finali β e γ sono distinguibili, in colonna c
- lo stato α (non finale) è distinguibile da ogni altro stato non finale (δ , ε e ζ), in colonna b e c
- lo stato δ (non finale) è distinguibile dagli stati non finali ε e ζ , in colonna c
- gli stati (non finali) ε e ζ sono distinguibili in colonna a , dove l'equivalenza $\gamma \sim \delta$ non sussiste

Pertanto quella data sopra è la tabella dei stati dell'automa minimo A'' , con 6 stati. Si lascia al lettore il compito di tracciarne il grafo stato-transizione.

- (c) Per ottenere l'espressione regolare R , equivalente all'automa A e non ambigua, si può per esempio procedere in modo algoritmico, ricavando R dall'automa deterministico (e minimo) A'' mediante il metodo dell'eliminazione dei nodi (Brozowski); si lascia il compito al lettore. In alternativa si può ricorrere al metodo delle equazioni linguistiche lineari a destra.

2 Grammatiche libere e automi a pila 20%

1. Le operazioni p ($= producer$) e c ($= consumer$) inseriscono e tolgono, rispettivamente, un elemento da un tampone (buffer) di dimensione illimitata; all'inizio il tampone contiene un solo elemento. Una sequenza di operazioni $x \in \{c, p\}^*$ (eventualmente nulla) è valida se non provoca "underflow" del tampone, ossia se non tenta di togliere un elemento quando il tampone è vuoto. Ecco alcuni esempi di stringhe valide:

$\varepsilon \quad p \quad pc \quad pp \quad cpc \quad cpp$

Ed ecco due esempi di stringhe non valide:

$cc \quad pccc$

Il linguaggio L è l'insieme delle sequenze valide.

Si risponda alle domande seguenti:

- (a) Si descriva a parole il funzionamento di un automa a pila A che riconosce L .
- (b) Per definire il linguaggio L si scriva una grammatica libera G , in forma non estesa (BNF) e preferibilmente non ambigua.
- (c) (facoltativo) Ci sono due tamponi (buffer), e due coppie produttore-consumatore p_1, c_1 e p_2, c_2 : la coppia p_1, c_1 opera sul primo tampone, la coppia p_2, c_2 sul secondo. Una sequenza di operazioni di entrambe le coppie (come per esempio $p_2 c_2 c_1 c_2$) è valida se non causa "underflow" in nessuno dei due tamponi. Si riconsideri la domanda (a) per il linguaggio delle sequenze valide.

Soluzione

- (a) Si descriva a parole il funzionamento di un automa a pila A che riconosce L .
Un automa a pila A riconosce facilmente il linguaggio L simulando il tampone per mezzo della pila. L'automa A inizia il calcolo con la pila contenente un simbolo P . Conviene scegliere come modalità di accettazione lo stato interno, e non la pila vuota, poiché il linguaggio accetta tutti i prefissi delle frasi. Di conseguenza, per riconoscere a pila vuota si dovrebbe programmare lo svuotamento della pila. Alla lettura di p , con P in cima alla pila, l'automa A aggiunge un altro simbolo P . Alla lettura di c l'automa A cancella un P dalla pila. Ogni stato è di riconoscimento.
- (b) Per definire il linguaggio L si scriva una grammatica libera G , in forma non estesa e preferibilmente non ambigua. Eccola:

$$G \left\{ \begin{array}{l} S \rightarrow p S S \\ S \rightarrow \varepsilon \\ S \rightarrow c \end{array} \right.$$

Le regole della grammatica G garantiscono che il numero di c sia minore o eguale al numero di p presenti nella stringa. Inoltre per la prima regola ogni c , tranne

eventualmente la prima, è preceduta da una corrispondente p . Tutto ciò vale per ogni prefisso della stringa, così evitando sempre di avere underflow.

La grammatica G è però ambigua, come si vede generando in due modi diversi la frase pc . Una versione non ambigua della grammatica G si può ottenere osservando che, supponendo il tampone inizialmente vuoto, le frasi del linguaggio avrebbero la struttura

$$p^* p^{n_1} c^{n_1} \underbrace{p^* p^{n_2} c^{n_2}}_{\text{fattore}} \dots p^* p^{n_k} c^{n_k} \quad \text{con } k \geq 0$$

e con $n_i \geq 1$ per ogni fattore, tranne per l'ultimo che può avere $n_k = 0$ a condizione che p^* non si annulli. Poiché il tampone inizialmente ha un elemento, si può anche inserire una c in qualsiasi punto della precedente espressione. L'effetto è di consentire anche frasi con la struttura

$$c p^* p^{n_1} c^{n_1} \dots$$

oppure

$$\begin{aligned} p^{n_1} c^{n_1} c p^* p^{n_2} c^{n_2} \dots p^* p^{n_k} c^{n_k} \\ p^{n_1} c^{n_1} p^{n_2} c^{n_2} c \dots p^* p^{n_k} c^{n_k} \end{aligned}$$

ecc.

Una possibile grammatica è:

$$G \left\{ \begin{array}{ll} S \rightarrow X \mid Y & \text{due tipi di strutture} \\ X \rightarrow PEX \mid PE \mid pP & \text{ultimo fattore può essere privo di } c \\ P \rightarrow pP \mid p & \\ E \rightarrow pEc \mid pc & \text{genera } p^n c^n \\ Y \rightarrow cX & c \text{ all'inizio} \\ Y \rightarrow ZcX & c \text{ dopo un prefisso } p^{n_1} c^{n_1} p^{n_2} c^{n_2} \dots \\ Z \rightarrow EZ \mid E & \end{array} \right.$$

- (c) (facoltativo) Ci sono due tamponi (buffer), e due coppie produttore-consumatore p_1, c_1 e p_2, c_2 : la coppia p_1, c_1 opera sul primo tampone, la coppia p_2, c_2 sul secondo. Una sequenza di operazioni di entrambe le coppie (come per esempio $p_2 c_2 c_1 c_2$) è valida se non causa “underflow” in nessuno dei due tamponi. Si riconsideri la domanda (a) per il linguaggio delle sequenze valide.

Una sola pila non basta per rappresentare lo stato dei due tamponi. Quindi è necessario un automa di tipo più complesso, come una macchina di Turing (o anche una macchina dotata di due contatori). Il linguaggio delle sequenze valide non è dunque libero, bensì dipendente dal contesto (di tipo 1 o 0).

2. Un testo marcato, ispirato a XML, può contenere tabelle e liste. Un esempio di tabella con due righe di tre e due elementi ciascuna, è il seguente:

```

<TABLE>
<TR>...<TD>...<TD>...
<TR>...<TD>...
<\ TABLE>

```

dove TR sta per Table Row, TD per Table Divider (ovvero separatore tra elementi di riga) e i puntini indicano i contenuti del testo. Un esempio di lista di tre elementi è il seguente (anche qui i puntini indicano i contenuti del testo):

```

<LIST>
<ITEM>...<ITEM>...<ITEM>...
<\ LIST>

```

Naturalmente il numero di righe e di elementi della tabella, e di elementi della lista, è arbitrario ma non nullo.

Si impongono le seguenti regole di composizione dei due costrutti:

- un elemento di tabella può essere di tipo lista o testo
- un elemento di lista può essere di tipo tabella o testo
- un elemento di tabella non può essere di tipo tabella
- un elemento di lista non può essere di tipo lista

Per modellare il testo si usi il simbolo terminale pt (che sta per *plain text*), che non è necessario espandere ulteriormente.

Si scriva una grammatica in forma estesa (EBNF) per tale linguaggio, verificando che essa non sia ambigua.

Soluzione

Si scriva una grammatica in forma estesa (EBNF) per tale linguaggio, verificando che essa non sia ambigua.

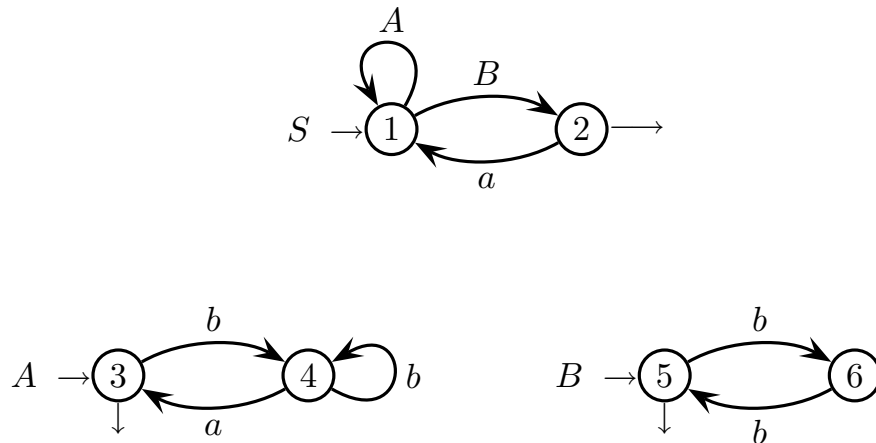
Per brevità, ridenominiamo i caratteri terminali come $\{t, t', r, d, l, l', i, p_t\}$.

Una possibile grammatica è:

$$G \left\{ \begin{array}{ll} S \rightarrow T \mid L & \\ T \rightarrow tE_T(dE_T)^+(rE_T(dE_T)^+)^*t' & \text{tabella} \\ E_T \rightarrow L \mid p_t & \text{elemento di tabella} \\ L \rightarrow l(iE_L)^+l' & \\ E_T \rightarrow T \mid p_t & \text{elemento di lista} \end{array} \right.$$

3 Analisi sintattica e parsificatori 20%

1. È data la rete di automi in figura:

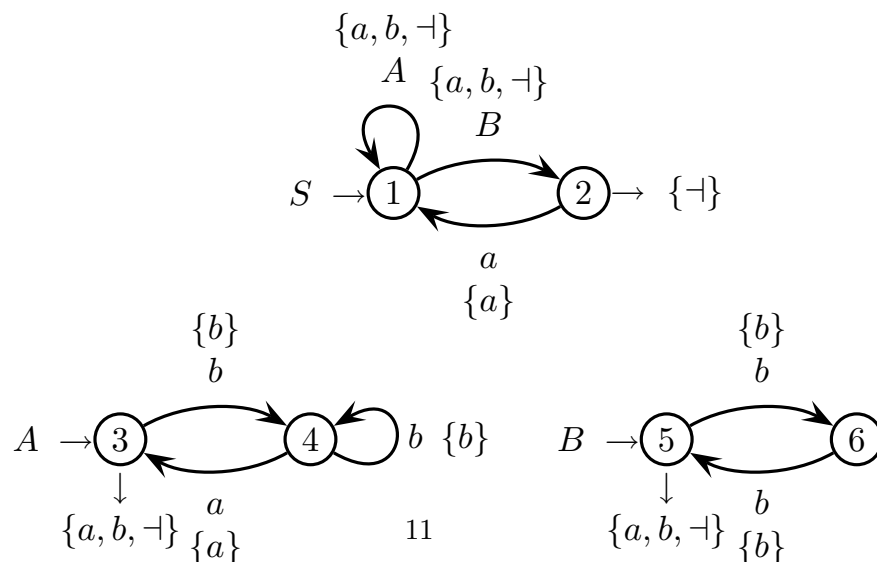


Si risponda alle domande seguenti:

- Si calcolino gli insiemi guida $LL(1)$ (scrivendone il valore calcolato nella figura in tutti i punti rilevanti) e si spieghi se la grammatica è $LL(1)$.
- Qualora la grammatica non risultasse $LL(1)$, si costruisca una grammatica equivalente $LL(1)$.

Soluzione

- Si calcolino gli insiemi guida $LL(1)$ (scrivendone il valore calcolato nella figura in tutti i punti rilevanti) e si spieghi se la grammatica è $LL(1)$. Ecco gli insiemi guida (solo dove serve):



La grammatica non è di tipo $LL(1)$: per esempio gli insiemi guida degli archi uscenti dallo stato a hanno l'elemento b in comune. Si vede facilmente che la grammatica non è $LL(l)$ per nessun $k \geq 1$.

- (b) Qualora la grammatica non risultasse $LL(1)$, si costruisca una grammatica equivalente $LL(1)$.

Come dimostrato prima, effettivamente la grammatica non è $LL(1)$. Il linguaggio però è regolare poiché la grammatica non contiene derivazioni ricorsive. Ecco l'espressione regolare del linguaggio per ogni macchina della rete:

$$S = (A \mid B a)^* B \quad A = (b^+ a)^* \quad B = (b b)^*$$

Sostituendo nella prima equazione le altre si ottiene la soluzione:

$$S = ((b^+ a)^* \mid (b b)^* a)^* (b b)^*$$

Ora si può calcolare un automa finito deterministico equivalente con il metodo di Berry-Sethi, da cui poi si ottiene la grammatica lineare a destra, che risulta $LL(1)$ per costruzione. Si lascia il compito al lettore.

2. È data la seguente grammatica G (assioma S):

$$G \left\{ \begin{array}{l} S \rightarrow a X \\ X \rightarrow Y b \\ Y \rightarrow b A \\ Y \rightarrow B a \\ A \rightarrow a B a \\ A \rightarrow a \\ B \rightarrow b A b \\ B \rightarrow b \end{array} \right.$$

Si risponda alle domande seguenti:

- (a) Si analizzi la stringa ab con il metodo Earley, riportando il calcolo nella tabella predisposta.
- (b) Si analizzi la stringa $abab$ con il metodo Earley, riportando il calcolo nella tabella predisposta.
- (c) Si specifichi il linguaggio generato dalla grammatica G .
- (d) (facoltativa) Si dica se la grammatica G sia ambigua, e se lo è si dica se il linguaggio generato da G sia inerentemente ambiguo, spiegandone il motivo; altrimenti si fornisca una grammatica G' equivalente a G ma non ambigua.

Schema di simulazione dell'algoritmo di Earley									
stato 0	pos. <i>a</i>	stato 1	pos. <i>b</i>	stato 2	pos. <i>a</i>	stato 3	pos. <i>b</i>	stato 4	

Soluzione

- (a) La stringa ab è l'unica non ambigua, pertanto viene costruito un solo albero sintattico.
- (b) L'analisi parte dalla precedente; la stringa $abab$ è ambigua e ha due alberi sintattici.
- (c) Il linguaggio $L = L(G)$ è regolare e si formula facilmente così:

$$L = \{ (ab)^n \mid n \geq 1 \} = (ab)^+$$

- (d) La grammatica G è ambigua, ma il linguaggio L non è inerentemente ambiguo. Una semplice grammatica G' non ambigua è la seguente:

$$G' \left\{ \begin{array}{l} S \rightarrow CS \mid C \\ C \rightarrow ab \end{array} \right.$$

4 Traduzione e analisi semantica 20%

1. Dato il linguaggio $\{a^n b \mid n \geq 0\}$, che rappresenta l'intero $n \geq 0$, si considerino le due funzioni di traduzione seguenti, che calcolano il quoziente intero e il resto della divisione per tre del numero dato:

$$\tau_1(a^n b) = q^{n \div 3} r^{n \bmod 3}$$

$$\tau_2(a^n b) = r^{n \bmod 3} q^{n \div 3}$$

dove gli operatori “ $\div$ ” e “ $\bmod$ ” calcolano il quoziente intero e il resto della divisione, rispettivamente.

Esempio:

$$\tau_1(a a a a a b) = q r r$$

$$\tau_2(a a a a a b) = r q q$$

Si risponda alle domande seguenti:

- (a) Si scriva uno schema di traduzione sintattico G' (o grammatica di traduzione) più semplice possibile per calcolare la funzione τ_1 .
- (b) Si scriva uno schema di traduzione sintattico G'' (o grammatica di traduzione) più semplice possibile per calcolare la funzione τ_2 .

Soluzione

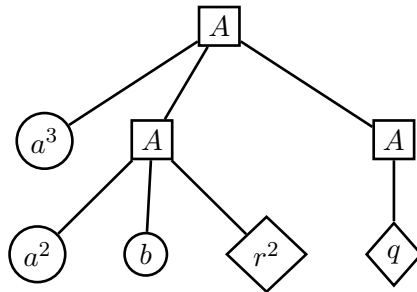
- (a) La traduzione τ_1 è razionale. Ecco lo schema G' , con regole di tipo lineare a destra (assioma A):

$$G' \left\{ \begin{array}{l} A \rightarrow a a a A \{q\} \\ A \rightarrow R \\ R \rightarrow a b \{r\} \\ R \rightarrow a a b \{r r\} \\ R \rightarrow b \end{array} \right.$$

- (b) La traduzione τ_2 è libera (un automa a stati finiti non basta). Ed ecco lo schema G'' (assioma A):

$$G' \left\{ \begin{array}{l} A \rightarrow a a a A Q \\ A \rightarrow b \\ A \rightarrow a b \{ r \} \\ R \rightarrow a a b \{ r r \} \\ Q \rightarrow q \end{array} \right.$$

Eccone un albero sintattico, per la traduzione $\tau_2 (a^5 b) = r^2 q$:



2. Tramite una grammatica con attributi, si modelli la traduzione della struttura di controllo “repeat ... until” in istruzioni di salto condizionato a etichette del programma. È data la sintassi G seguente:

$$G \left\{ \begin{array}{ll} S_0 \rightarrow L & \\ L \rightarrow F \text{ ‘;’ } L & L \text{ è una lista di frasi o istruzioni} \\ L \rightarrow F & F \text{ è una frase o istruzione} \\ F \rightarrow A & A \text{ è un assegnamento} \\ F \rightarrow \text{repeat } L \text{ until } C & C \text{ è una condizione} \end{array} \right.$$

Si usino allo scopo gli attributi seguenti:

tr	codice risultante dalla traduzione di un costrutto - si possono concatenare traduzioni di diversi costrutti mediante l'operatore “.”
n	attributo intero delle istruzioni utilizzabile per generare le etichette bersaglio dei salti
rc	attributo della condizione C - indica il registro dove porre il risultato della valutazione della condizione

Si utilizzi l'istruzione seguente:

jump-if-false r, e

per codificare il salto all'istruzione di etichetta “ e ” se il valore nel registro “ r ” è falso. Si faccia uso della funzione *nuovo*, che a ogni invocazione restituisce un diverso valore intero per generare le etichette.

Si risponda alle domande seguenti:

- Si definiscano compiutamente gli attributi necessari e si scrivano le funzioni semantiche per il calcolo degli attributi (si vedano le tabelle predisposte).
- Si mostri il risultato della traduzione sull'esempio seguente:

sorgente	traduzione da scrivere
$a := 1;$ $n := 0;$ repeat $a := a * 2;$ $n := n + 1$ until $2 * a > k$	

attributi da usare per la grammatica

tipo	nome	(non)terminali	dominio	significato
------	------	----------------	---------	-------------

già dati nel testo dell'esercizio

	<i>tr</i>		stringa	codice risultante dalla traduzione di un costrutto
	<i>n</i>		intero	attributo per generare le etichette dei salti
	<i>rc</i>		stringa	attributo di <i>C</i>

eventualmente da estendere e / o aggiungerne di nuovi

--	--	--	--	--

<i>sintassi</i>	<i>funzioni semantiche</i> (punto (a))
$S_0 \rightarrow L_1$	
$L_0 \rightarrow F_1; L_2$	
$L_0 \rightarrow F_1$	
$f_0 \rightarrow A$	
$F_0 \rightarrow \text{repeat } L_1 \text{ until } C_2$	

Soluzione

(a) Ecco le funzioni semantiche:

regola	funzione semantica
$S_0 \rightarrow L_1$	$tr_0 \leftarrow tr_1$
$L_0 \rightarrow F_1; L_2 \text{ ‘,’ } L_2$	$tr_0 \leftarrow tr_1 \cdot tr_2$
$L_0 \rightarrow F_1$	$tr_0 \leftarrow tr_1$
$F_0 \rightarrow \text{repeat } L_1 \text{ until } C_2$	$tr_0 \leftarrow \text{‘}e\text{’} \cdot n_0 \text{ ‘:’ } tr_1 \text{ jmp-if-false } rc_2 \text{ ‘},e\text{’ } n_0$
	$n_1 \leftarrow nuovo$

(b) ***DA FARE***